University
Mohammed VI
Polytechnic

College of
Computing

Lab 5:Normalization

Data Management Course

UM6P College of Computing

Professor: Karima Echihabi **Program:** Computer Engineering
Session: Fall 2026

Team Information

Team Name	los primeros
Member 1	Sarah Bouchmal
Member 2	Douaa DYA
Member 3	Fatimazahra Benjaa
Member 4	Douaa Charafi

This is only a template. Feel free to adapt the structure according to the instructions for each lab.

1 Introduction

Relational schemas may contain redundancies that lead to several problems, such as update, insertion, and deletion anomalies. To minimize these problems, we apply normalization techniques to ensure that our relational schemas satisfy appropriate normal forms, such as BCNF or 3NF.

This deliverable focuses on database normalization techniques, including:
-Identifying the functional dependencies (FDs) in relational schemas.
-Analyzing relational schemas to check whether they satisfy BCNF and 3NF.
-Decomposing relational schemas when necessary.
-Verifying lossless decomposition and dependency preservation.

2 Requirements

– Analyze the relational schema from Lab 4 and identify any violations of BCNF.

– Determine the candidate keys for each relation in Question 2.

– Check whether each relation in Question 2 satisfies BCNF.

– Identify the highest normal form satisfied by each relation.

– Decompose relations that violate BCNF into smaller relations.

– Verify whether the decompositions are lossless and preserve functional dependencies.

3 Methodology

Roles and Contributions

- **Douaa Dya:** Completed Question 1, which involved checking BCNF for the relations from Lab 4, and all questions related to Equipment Inventory.
- **Douae Charafi:** Completed all questions related to Project Participation.
- **Sarah Bouchmal:** Completed all questions related to Reservation and Project Information and worked alongside Fatima Zahra on Supplier Offer
- **Fatima Zahra Benjaa:** Completed all questions related Equipment Ceritifcation Requirement and worked alongside Sarah on Supplier Offer

4 Implementation & Results

4.1 Question1:

-There are no BCNF violations in the relational schema from Lab 4 because, for every non-trivial functional dependency $X \rightarrow Y$ in each relation, the left-hand side X is a superkey of that relation. Therefore, all relations satisfy BCNF.

4.2 Question 2

a) **Equipment Inventory**

We have:

$$\text{EquipmentInventory}(\text{serialNo}, \text{modelId}, \text{manufacturer}, \text{labId})$$

$$F = \{\text{serialNo} \rightarrow \{\text{modelId}, \text{labId}\}, \text{modelId} \rightarrow \text{manufacturer}\}$$

1. The Candidate Key(s):

$$\text{serialNo}^+ = \{\text{serialNo}, \text{modelId}, \text{labId}, \text{manufacturer}\} = \text{EquipmentInventory}$$

Therefore,serialNo is a superkey and, since it is minimal, a candidate key.
-The attributes modelId, manufacturer, and labId are not candidate keys individually, since their closures do not contain all attributes of the relation.

Thus, the only candidate key is serialNo. **2. BCNF violation verification:**

-We check for every non-trivial FD ($X \rightarrow Y$) in F if X is a superkey :

serialNo is a superkey. But, modelid is not, because:

$$\text{modelId}^+ = \{\text{modelId}, \text{manufacturer}\} \neq \text{EquipmentInventory}$$

So, the relation EquipmentInventory violates BCNF and The FD responsible for the violation is ($modelId \rightarrow manufacturer$) **3. The strongest Normal Form:**

-We check for every non-trivial FD ($X \rightarrow Y$) in F if Y is a part of a candidate key:

-Since the only candidate we have is serialNo and manufacturer is not a part of serialNo. Then, the relation EquipmentInventory is not 3NF. Therefore, the relation EquipmentInventory is 1NF. **4. BCNF decomposition:**

The violating FD is : $modelId \rightarrow manufacturer$

we create two relations R1 and R2. The first one is EquipmentInventory without manufacturer attribute, the second one contains the attributes modelId and manufacturer.

$$R1(serialNo, modelId, labId)$$

$$R2(modelId, manufacturer)$$

5. Verification of lossness decomposition:
-We check the intersection of the two tables and see if it determines all the attributes of one of the two relations.

$$R_1 \cap R_2 = \{\text{modelId}\}$$

We have : $modelId \rightarrow manufacturer$ and $modelId \rightarrow modelId$

Therefore, $R_1 \cap R_2 \rightarrow R_1$.
The decomposition is lossless.

6. Dependency Preservation:

We check whether:

$$(F_1 \cup F_2)^+ = F^+$$

We have:

$$F_1 = \{\text{serialNo} \rightarrow \text{modelId}, \quad \text{serialNo} \rightarrow \text{labId}\}$$

$$F_2 = \{\text{modelId} \rightarrow \text{manufacturer}\}$$

Therefore:

$$F_1 \cup F_2 = \left\{ \begin{array}{l} \text{serialNo} \rightarrow \text{modelId}, \\ \text{serialNo} \rightarrow \text{labId}, \\ \text{modelId} \rightarrow \text{manufacturer} \end{array} \right\}$$

By transitivity, we also obtain:

$$\text{serialNo} \rightarrow \text{manufacturer}$$

$$\boxed{(F_1 \cup F_2)^+ = F^+}$$

Therefore, the decomposition is **dependency-preserving**.

b) **Project Participation**

Consider the relation

$$ProjectParticipation(\text{pid}, \text{projectCode}, \text{role}, \text{piPid})$$

with the following functional dependencies:

$$F = \left\{ \begin{matrix} (\text{pid}, \text{projectCode}) \rightarrow \text{role}, \\ \text{projectCode} \rightarrow \text{piPid} \end{matrix} \right\}$$

1. Candidate key

We have:

$$(\text{pid}, \text{projectCode})^+ = \{\text{pid}, \text{projectCode}, \text{role}, \text{piPid}\} = \text{ParticipationProject}$$

The closure contains every attribute of R. Neither pid nor projectCode can be removed: neither attribute is determined by any given dependency. Thus, the candidate key is

$$\boxed{\{\text{pid}, \text{projectCode}\}}.$$

2. BCNF verification

A relation is in BCNF if the determinant of every non-trivial functional dependency is a superkey.

For

$$(\text{pid}, \text{projectCode}) \rightarrow \text{role},$$

the determinant is a candidate key, so this dependency does not violate BCNF.

For

$$\text{projectCode} \rightarrow \text{piPid},$$

we have

$$\text{projectCode}^+ = \{\text{projectCode}, \text{piPid}\}.$$

This closure does not contain pid or role; therefore, projectCode is not a superkey. This dependency violates BCNF, so R is not in BCNF. **3. Strongest Normal Form**

Since R violates BCNF, we check whether it satisfies 3NF.

A relation is in 3NF if, for every non-trivial functional dependency $X \rightarrow A$, either X is a superkey or A is a prime attribute.

Consider the functional dependency

$$\text{projectCode} \rightarrow \text{piPid}.$$

We know that projectCode is not a superkey. Furthermore, piPid is not a prime attribute because it does not belong to the candidate key (pid, projectCode).

Therefore, this functional dependency violates 3NF, and R is not in 3NF. Therefore, the strongest normal form satisfied by the original relation is

$$\boxed{\text{1NF}}.$$

4.BCNF decomposition

Decompose R using the violating dependency projectCode $\rightarrow$ piPid:

$$R_1(\text{projectCode}, \text{piPid})$$

and

$$R_2(\text{pid}, \text{projectCode}, \text{role}).$$

In R_1, projectCode determines all attributes and is a candidate key. Thus, R_1 is in BCNF.

In R_2, the dependency

$$(\text{pid}, \text{projectCode}) \rightarrow \text{role}$$

holds, and (pid, projectCode) is a candidate key. Thus, R_2 is also in BCNF.

5. Lossless join

The common attribute is

$$R_1 \cap R_2 = \{\text{projectCode}\}.$$

Because

$$\text{projectCode} \rightarrow \text{piPid},$$

the common attribute determines every attribute of R_1. Therefore, the decomposition is lossless.

6. Dependency preservation

The dependency

$$(\text{pid}, \text{projectCode}) \rightarrow \text{role}$$

is preserved in R_2, while

$$\text{projectCode} \rightarrow \text{piPid}$$

is preserved in R_1. Hence, all the original functional dependencies are preserved without requiring a join.

c) **Equipment Certification Requirement**

Consider the relation

$$\mathit{EquipmentCertification}(\text{modelId}, \text{certCode}, \text{certTitle}, \text{issuingAuthority})$$

with the functional dependency

$$\text{certCode} \rightarrow \{\text{certTitle}, \text{issuingAuthority}\}$$

and where the combination (modelId, certCode) identifies an equipment-model requirement.

1. Identify the candidate key(s)

The closure of {modelId, certCode} is

$$\begin{aligned}(\text{modelId}, \text{certCode})^+ &= \{\text{modelId}, \text{certCode}\} \\ &\quad \cup \{\text{certTitle}, \text{issuingAuthority}\} \\ &= \{\text{modelId}, \text{certCode}, \text{certTitle}, \text{issuingAuthority}\}.\end{aligned}$$

Therefore, {modelId, certCode} is a superkey.

Moreover,

$$\text{modelId} \notin \text{certCode}^+$$

and

$$\text{certCode} \notin \text{modelId}^+.$$

Hence, neither {modelId} nor {certCode} is a superkey. The combination {modelId, certCode} is therefore minimal and is a candidate key.

It is the only candidate key because neither modelId nor certCode can be derived from the other attributes, and both must belong to every candidate key.

Thus, the only candidate key is

$$\boxed{\{\text{modelId}, \text{certCode}\}}.$$

2. Does the relation satisfy BCNF?

No, the relation does not satisfy BCNF.

The functional dependency

$$\text{certCode} \rightarrow \{\text{certTitle}, \text{issuingAuthority}\}$$

is non-trivial because

$$\{\text{certTitle}, \text{issuingAuthority}\} \not\subseteq \{\text{certCode}\}.$$

However, certCode is not a superkey because

$$\text{certCode}^+ = \{\text{certCode}, \text{certTitle}, \text{issuingAuthority}\}$$

does not contain modelId.

Therefore, the relation violates BCNF.

3. Identify the functional dependency responsible for the BCNF violation and determine the strongest normal form satisfied

The functional dependency responsible for the BCNF violation is

$$\boxed{\text{certCode} \rightarrow \{\text{certTitle}, \text{issuingAuthority}\}}.$$

Its determinant, certCode, is not a superkey because its closure does not contain all the attributes of R.

The relation also violates 3NF because neither certTitle nor issuingAuthority is a prime attribute; that is, neither belongs to any candidate key.

Since all attributes are atomic, the relation satisfies 1NF.

Therefore, the strongest normal form satisfied is

$$\boxed{\text{1NF}}.$$

4. Decompose the relation into BCNF relations

We decompose $EquipmentCertification$ using the functional dependency

$$\text{certCode} \rightarrow \{\text{certTitle}, \text{issuingAuthority}\}.$$

The resulting relations are

$$\boxed{R_1(\text{modelId}, \text{certCode})}$$

and

$$\boxed{R_2(\text{certCode}, \text{certTitle}, \text{issuingAuthority})}.$$

For R_1, the functional dependency is

$$\text{modelId}, \text{certCode} \rightarrow \{\text{modelId}, \text{certCode}\}.$$

This dependency is trivial, and the combination (modelId, certCode) is a candidate key. Therefore, R_1 is in BCNF.

For R_2, the functional dependency is

$$\text{certCode} \rightarrow \{\text{certTitle}, \text{issuingAuthority}\}.$$

The attribute certCode is a candidate key of R_2. Therefore, R_2 is also in BCNF.
Hence, the decomposition produces two BCNF relations.

5. Determine whether the decomposition is lossless

The intersection of the two relations is

$$R_1 \cap R_2 = \{\text{certCode}\}.$$

Since

$$\text{certCode} \rightarrow \{\text{certCode}, \text{certTitle}, \text{issuingAuthority}\},$$

the intersection functionally determines all the attributes of R_2.
Therefore, the decomposition is lossless.

6. Are the relevant functional dependencies preserved?

Yes, the relevant functional dependencies are preserved.
The projection of the functional dependencies onto R_1 contains

$$F_1 = \{(\text{modelId}, \text{certCode}) \rightarrow (\text{modelId}, \text{certCode})\}.$$

The projection onto R_2 contains

$$F_2 = \{\text{certCode} \rightarrow (\text{certTitle}, \text{issuingAuthority})\}.$$

The original non-trivial functional dependency is preserved in F_2. The trivial dependency in F_1 is automatically satisfied.
Thus, the decomposition is dependency-preserving.

—

d) **Supplier Offer**

Consider the relation

$$SupplierOffer(\text{suppId}, \text{consId}, \text{labId}, \text{supplierName}, \text{unitPrice})$$

with the functional dependencies

$$\text{suppId} \rightarrow \text{supplierName}$$

and

$$(\text{suppId}, \text{consId}, \text{labId}) \rightarrow \text{unitPrice}.$$

1. Identify the candidate key(s)

The closure of $\{\text{suppId}, \text{consId}, \text{labId}\}$ is

$$\begin{aligned}
&(\text{suppId}, \text{consId}, \text{labId})^+ \\
&= \{\text{suppId}, \text{consId}, \text{labId}\} \\
&\quad \cup \{\text{supplierName}\} \\
&\quad \cup \{\text{unitPrice}\} \\
&= \{\text{suppId}, \text{consId}, \text{labId}, \text{supplierName}, \text{unitPrice}\}.
\end{aligned}$$

Therefore, $\{\text{suppId}, \text{consId}, \text{labId}\}$ is a superkey.

It is also minimal because

$$\text{suppId} \notin (\text{consId}, \text{labId})^+,$$

$$\text{consId} \notin (\text{suppId}, \text{labId})^+,$$

and

$$\text{labId} \notin (\text{suppId}, \text{consId})^+.$$

Consequently, none of the three attributes can be removed while retaining a superkey.

Thus, the candidate key is

$$\boxed{\{\text{suppId}, \text{consId}, \text{labId}\}}.$$

It is the only candidate key because none of the attributes suppId, consId, and labId appears on the right-hand side of a non-trivial functional dependency. Therefore, all three must belong to every candidate key.

In particular, although

$$\text{suppId} \rightarrow \text{supplierName},$$

suppId is not a superkey because its closure does not contain consId, labId, or unitPrice.

2. Does the relation satisfy BCNF?

No, the relation does not satisfy BCNF.

The functional dependency

$$\text{suppId} \rightarrow \text{supplierName}$$

is non-trivial because

$$\{\text{supplierName}\} \not\subseteq \{\text{suppId}\}.$$

However, suppId is not a superkey because

$$\text{suppId}^{+} = \{\text{suppId}, \text{supplierName}\}$$

does not contain all the attributes of R.

Therefore, the relation violates BCNF.

3. Identify the functional dependency responsible for the BCNF violation and determine the strongest normal form satisfied

The functional dependency responsible for the violation is

$$\boxed{\text{suppId} \rightarrow \text{supplierName}}.$$

Its determinant, suppId, is not a superkey.

The relation also violates 3NF because supplierName is not a prime attribute: it does not belong to any candidate key.

Since all attributes are atomic, the relation satisfies 1NF.

Therefore, the strongest normal form satisfied is

$$\boxed{\text{1NF}}.$$

4. Decompose the relation into BCNF relations

We decompose $SupplierOffer$ using the functional dependency

$$\text{suppId} \rightarrow \{\text{supplierName}\}.$$

The resulting relations are

$$\boxed{R_1(\text{suppId}, \text{supplierName})}$$

and

$$\boxed{R_2(\text{suppId}, \text{consId}, \text{labId}, \text{unitPrice})}.$$

For R_1, the functional dependency is

$$\text{suppID} \rightarrow \{\text{supplierName}\}.$$

This dependency is trivial. Therefore, R_1 is in BCNF.

For R_2, the functional dependency is

$$(\text{suppId}, \text{consId}, \text{labId}) \rightarrow \text{unitPrice}.$$

The attribute suppId, consId, labId is a candidate key of R_2. Therefore, R_2 is also in BCNF.

Hence, the decomposition produces two BCNF relations.

5. Determine whether the decomposition is lossless

The intersection of the two relations is

$$R_1 \cap R_2 = \{\text{suppId}\}.$$

Since

$$\text{suppId} \rightarrow \{\text{supplierName}\}$$

the intersection functionally determines all the attributes of R_1.

Therefore, the decomposition is lossless.

6. Are the relevant functional dependencies preserved?

Yes, the relevant functional dependencies are preserved.
The projection of the functional dependencies onto R_1 contains

$$F_1 = \{\text{suppId} \rightarrow \text{supplierName}\}.$$

The projection onto R_2 contains

$$F_2 = \{(\text{suppId}, \text{consId}, \text{labId}) \rightarrow \text{unitPrice}.\}$$

The original non-trivial functional dependencies are preserved in F_1 and F_2
Thus, the decomposition is dependency-preserving.

e) **Reservation and Project Information**

Consider the relation

$$ReservationProject(\text{resId}, \text{projectId}, \text{projectTitle}, \text{madeByPid}, \text{status})$$

with the functional dependencies

$$\text{resId} \rightarrow \{\text{projectCode}, \text{madebyPid}, \text{status}\}$$

and

$$\text{projectCode} \rightarrow \text{projectTitle}.$$

1. Identify the candidate key(s)

The closure of {resId} is

$$\begin{aligned}&(\text{resId})^+\\ &= \{\text{resID}\}\\ &\quad \cup \{\text{projectCode}, \text{madebyPid}, \text{status}\}\\ &\quad \cup \{\text{projectTitle}\}(FD2)\\ &= \{\text{resId}, \text{projectCode}, \text{madebyPid}, \text{projectTitlE}, \text{status}\}.\end{aligned}$$

Thus, the candidate key is

$$\boxed{\text{resId}}$$

because it is determines the whole relation + it is minimal

2. Does the relation satisfy BCNF?

No, the relation does not satisfy BCNF.
The functional dependency

$$\text{projectCode} \rightarrow \text{projectTitle}$$

is non-trivial because

$$\{\text{projectTitle}\} \not\subseteq \{\text{projectCode}\}.$$

However, projectCode is not a superkey because

$$\text{projectCode}^+ = \{\text{projectCode}, \text{projectTitle}\}$$

does not contain all the attributes of R.
Therefore, the relation violates BCNF.

3. Identify the functional dependency responsible for the BCNF violation and determine the strongest normal form satisfied

The functional dependency responsible for the violation is

$$\boxed{\text{projectCode} \rightarrow \text{projectTitle}}.$$

The relation also violates 3NF because projectTitle is not a prime attribute: it does not belong to any candidate key.

Since all attributes are atomic, the relation satisfies 1NF.

Therefore, the strongest normal form satisfied is (keeping in mind that we have not yet studied the criteria for 2NF)

$$\boxed{\text{1NF}}.$$

4. Decompose the relation into BCNF relations

We decompose *ReservationProject* using the functional dependency

$$\text{projectCode} \rightarrow \{\text{projectTitle}\}.$$

The resulting relations are

$$\boxed{R_1(\text{projectCode}, \text{projectTitle})}$$

and

$$\boxed{R_2(\text{resId}, \text{projectCode}, \text{madeByPid}, \text{status})}.$$

For R_1, the functional dependency is

$$\text{projectCode} \rightarrow \{\text{projectTitle}\}.$$

This dependency is trivial, because projectCode is a candidate key in R_1 Therefore, R_1 is in BCNF.

For R_2, the functional dependency is

$$\text{resId} \rightarrow \{\text{projectCode}, \text{madeByPid}, \text{status}\}.$$

The attribute resId is a candidate key of R_2. Therefore, R_2 is also in BCNF.

Hence, the decomposition produces two BCNF relations.

5. Determine whether the decomposition is lossless

The intersection of the two relations is

$$R_1 \cap R_2 = \{\text{projectCode}\}.$$

Since

$$\text{projectCode} \rightarrow \{\text{projectTitle}\}$$

the intersection functionally determines all the attributes of R_1.
Therefore, the decomposition is lossless.

6. Are the relevant functional dependencies preserved?

Yes, the relevant functional dependencies are preserved.
The projection of the functional dependencies onto R_1 contains

$$F_1 = \{\text{projectCode} \rightarrow \text{projectTitle}\}.$$

The projection onto R_2 contains

$$F_2 = \{\text{resId} \rightarrow \{\text{projectCode}, \text{madeByPid}, \text{status}\}\}$$

The original non-trivial functional dependencies are preserved in F_1 and F_2
Thus, the decomposition is dependency-preserving.

5 Discussion

The implementation was pretty straight-forward. We did not have much trouble finding out which form fits the studied relation albeit we had some difficulty decomposing into bCNF relations.

6 Conclusion

This deliverable contains our work on this lab + a thorough walkthrough each and every step taken to arrive to the conclusions we arrived to.